

✓ RAG with ChromaDB – Simple Google Colab Demo

This notebook demonstrates Retrieval Augmented Generation (RAG) using ChromaDB.

```
# Install necessary libraries: chromadb for the vector database and sentence-transformers for generating embeddings.
!pip install chromadb sentence-transformers

Requirement already satisfied: chromadb in /usr/local/lib/python3.12/dist-packages (from requests-ssl[chromadb])
Downloading chromadb-1.4.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (21.1 MB)
-----
21.1/21.1 MB 55.9 MB/s eta 0:00:00
Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
-----
278.2/278.2 kB 20.0 MB/s eta 0:00:00
Downloading build-1.4.0-py3-none-any.whl (24 kB)
Downloading kubernetes-35.0.0-py2.py3-none-any.whl (2.0 MB)
-----
2.0/2.0 MB 61.7 MB/s eta 0:00:00
Downloading onnxruntime-1.24.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (17.1 MB)
-----
17.1/17.1 MB 61.3 MB/s eta 0:00:00
Downloading opentelemetry_exporter_otlp_proto_grpc-1.39.1-py3-none-any.whl (19 kB)
Downloading opentelemetry_exporter_otlp_proto_common-1.39.1-py3-none-any.whl (18 kB)
Downloading opentelemetry_proto-1.39.1-py3-none-any.whl (72 kB)
-----
72.5/72.5 kB 4.8 MB/s eta 0:00:00
Downloading opentelemetry_sdk-1.39.1-py3-none-any.whl (132 kB)
-----
132.6/132.6 kB 11.6 MB/s eta 0:00:00
Downloading opentelemetry_api-1.39.1-py3-none-any.whl (66 kB)
-----
66.4/66.4 kB 3.6 MB/s eta 0:00:00
Downloading opentelemetry_semantic_conventions-0.60b1-py3-none-any.whl (219 kB)
-----
220.0/220.0 kB 15.7 MB/s eta 0:00:00
Downloading posthog-5.4.0-py3-none-any.whl (105 kB)
-----
105.4/105.4 kB 8.1 MB/s eta 0:00:00
Downloading pybase64-1.4.3-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl (
-----
71.6/71.6 kB 5.8 MB/s eta 0:00:00
Downloading pypika-0.51.1-py2.py3-none-any.whl (60 kB)
-----
60.6/60.6 kB 5.3 MB/s eta 0:00:00
Downloading backoff-2.2.1-py3-none-any.whl (15 kB)
Downloading durationpy-0.10-py3-none-any.whl (3.9 kB)
Downloading pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Installing collected packages: pypika, durationpy, pyproject_hooks, pybase64, opentelemetry-proto, bcrypt, backoff, posthog, op
Attempting uninstall: opentelemetry-proto
  Found existing installation: opentelemetry-proto 1.37.0
  Uninstalling opentelemetry-proto-1.37.0:
    Successfully uninstalled opentelemetry-proto-1.37.0
Attempting uninstall: opentelemetry-exporter-otlp-proto-common
  Found existing installation: opentelemetry-exporter-otlp-proto-common 1.37.0
  Uninstalling opentelemetry-exporter-otlp-proto-common-1.37.0:
    Successfully uninstalled opentelemetry-exporter-otlp-proto-common-1.37.0
Attempting uninstall: opentelemetry-api
  Found existing installation: opentelemetry-api 1.37.0
  Uninstalling opentelemetry-api-1.37.0:
    Successfully uninstalled opentelemetry-api-1.37.0
Attempting uninstall: opentelemetry-semantic-conventions
  Found existing installation: opentelemetry-semantic-conventions 0.58b0
  Uninstalling opentelemetry-semantic-conventions-0.58b0:
    Successfully uninstalled opentelemetry-semantic-conventions-0.58b0
Attempting uninstall: opentelemetry-sdk
  Found existing installation: opentelemetry-sdk 1.37.0
  Uninstalling opentelemetry-sdk-1.37.0:
    Successfully uninstalled opentelemetry-sdk-1.37.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is tl
google-adk 1.23.0 requires opentelemetry-api<=1.37.0,>=1.37.0, but you have opentelemetry-api 1.39.1 which is incompatible.
google-adk 1.23.0 requires opentelemetry-sdk<=1.37.0,>=1.37.0, but you have opentelemetry-sdk 1.39.1 which is incompatible.
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-exporter-otlp-proto-common==1.37.0, but you have openteler
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-proto==1.37.0, but you have opentelemetry-proto 1.39.1 wh
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-sdk==1.37.0, but you have opentelemetry-sdk 1.39.1 which
opentelemetry-exporter-gcp-logging 1.11.0a0 requires opentelemetry-sdk<1.39.0,>=1.35.0, but you have opentelemetry-sdk 1.39.1
Successfully installed backoff-2.2.1 bcrypt-5.0.0 build-1.4.0 chromadb-1.4.1 durationpy-0.10 kubernetes-35.0.0 onnxruntime-1.24
```

```
documents = [
    "RAG stands for Retrieval Augmented Generation.",
    "ChromaDB is a vector database used to store embeddings.",
    "RAG reduces hallucinations by grounding answers using retrieved documents.",
    "Fine-tuning changes behavior, RAG injects knowledge at runtime.",
    "RAG retrieves relevant chunks before generation."
]
# Display the list of documents to be used.
documents

['RAG stands for Retrieval Augmented Generation.',
 'ChromaDB is a vector database used to store embeddings.',
```

```
'RAG reduces hallucinations by grounding answers using retrieved documents.',
'Fine-tuning changes behavior, RAG injects knowledge at runtime.',
'RAG retrieves relevant chunks before generation.']
```

```
# Import the SentenceTransformer model for generating embeddings.
from sentence_transformers import SentenceTransformer
# Import the chromadb library for interacting with ChromaDB.
import chromadb

# Initialize the SentenceTransformer model with 'all-MiniLM-L6-v2' for creating embeddings.
model = SentenceTransformer('all-MiniLM-L6-v2')
# Generate embeddings for each document in the 'documents' list.
embeddings = model.encode(documents)

# Initialize the ChromaDB client.
client = chromadb.Client()
# Create a new collection named 'rag_demo' in ChromaDB.
collection = client.create_collection('rag_demo')

# Iterate through each document and its corresponding embedding.
for i, doc in enumerate(documents):
    # Add each document, its ID, and its embedding (converted to a list) to the ChromaDB collection.
    collection.add(ids=[str(i)], documents=[doc], embeddings=[embeddings[i].tolist()])

# Print a confirmation message once documents are stored.
print('Documents stored in ChromaDB')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
```

```
modules.json: 100% 349/349 [00:00<00:00, 16.9kB/s]
```

```
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 6.10kB/s]
```

```
README.md: 10.5k/? [00:00<00:00, 822kB/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
```

```
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.12kB/s]
```

```
config.json: 100% 612/612 [00:00<00:00, 63.1kB/s]
```

```
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 125MB/s]
```

```
Loading weights: 100% 103/103 [00:00<00:00, 497.24it/s, Materializing param=pooler.dense.weight]
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
```

```
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |
```

```
Notes:
```

```
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
tokenizer_config.json: 100% 350/350 [00:00<00:00, 20.9kB/s]
```

```
vocab.txt: 232k/? [00:00<00:00, 5.98MB/s]
```

```
tokenizer.json: 466k/? [00:00<00:00, 17.0MB/s]
```

```
special_tokens_map.json: 100% 112/112 [00:00<00:00, 9.70kB/s]
```

```
config.json: 100% 190/190 [00:00<00:00, 18.9kB/s]
```

```
Documents stored in ChromaDB
```

```
# Define the query string.
query = 'How does RAG reduce hallucinations?'
# Generate the embedding for the query.
q_embed = model.encode([query])

# Perform a similarity search in the ChromaDB collection using the query embedding.
# Retrieve the top 2 most relevant results.
results = collection.query(query_embeddings=q_embed.tolist(), n_results=2)
# Extract and display the documents from the search results.
results['documents'][0]
```

```
['RAG reduces hallucinations by grounding answers using retrieved documents.',  
 'RAG stands for Retrieval Augmented Generation.']
```